

**Sistema de adquisición y análisis de señales en circuitos RLC mediante ESP32 y
procesamiento en Python**

Andrés David Nazarith Gómez 1031647255
Juan Manuel Gonzales Banguero 1112041197
Luis José Pinto González 184831687

Procesamiento Digital de Señales

Facultad de Ingeniería
Programa de Ingeniería Electrónica
UNIVERSIDAD SANTIAGO DE CALI
2026

Contenido

<i>RESUMEN</i>	3
INTRODUCCIÓN	3
PLANTEAMIENTO DEL PROBLEMA	4
JUSTIFICACIÓN	4
MARCO CONCEPTUAL	5
OBJETIVO GENERAL Y ESPECÍFICOS	6
METODOLOGÍA	7
RESULTADOS	8
POSIBLES MEJORAS Y TRABAJO FUTURO	22
CRONOGRAMA	24
PRESUPUESTO	25
Bibliografía	27

RESUMEN

Este proyecto propone el desarrollo de un sistema de adquisición y procesamiento digital de señales con soporte de una interfaz gráfica, aplicado al análisis dinámico de un circuito RLC. Mediante el uso de un microcontrolador ESP32 se realiza la captura de señales eléctricas en el dominio del tiempo, las cuales posteriormente son procesadas en Python para su análisis temporal y frecuencial.

La interfaz desarrollada permite visualizar la señal muestreada, así como la interacción del usuario con los distintos módulos de procesamiento digital, incluyendo el filtrado digital para la reducción de ruido, análisis espectral mediante la transformada de Fourier y la estimación de parámetros dinámicos como la frecuencia natural y el factor de amortiguamiento.

El objetivo del proyecto es automatizar el análisis experimental de circuitos eléctricos de segundo orden, y al mismo tiempo, facilitando el aprendizaje de estos sistemas, proporcionando a los estudiantes una herramienta que integre la adquisición, procesamiento digital y visualización de los resultados.

Palabras claves: Circuito RLC, ESP32, adquisición de datos, análisis de señales, interfaz gráfica, procesamiento digital.

Keywords: RLC circuit, data acquisition, signal analysis, graphical interface, Arduino Nano.

INTRODUCCIÓN

El análisis de señales eléctricas es fundamental en la formación del ingeniero electrónico, especialmente en el estudio de circuitos de segundo orden como los circuitos RLC. Tradicionalmente, este análisis se realiza utilizando instrumentos analógicos como osciloscopios, lo que puede limitar la automatización y el procesamiento posterior de los datos.

Con el avance de los sistemas embebidos y herramientas de programación, es posible integrar hardware y software para desarrollar herramientas que permitan no solo adquirir datos, sino analizarlos automáticamente. En este contexto, el procesamiento digital de señales se convierte en una herramienta esencial para el estudio de fenómenos como la respuesta transitoria, el comportamiento oscilatorio, la respuesta en frecuencia y la estimación de parámetros dinámicos a partir de datos experimentales.

Los circuitos RLC son un caso ideal para la aplicación de técnicas de procesamiento digital ya que su comportamiento puede ser analizado tanto en el dominio del tiempo

como en el dominio de la frecuencia. Mediante la transformada rápida de Fourier (FFT), filtros digitales y la identificación de sistemas, es posible estimar la frecuencia natural, factor de amortiguamiento y la influencia del ruido en la señal.

En este proyecto se desarrolla un sistema de adquisición, visualización y análisis digital de señales de un circuito RLC, utilizando el microcontrolador ESP32 para la toma de datos y Python para su procesamiento. La interfaz gráfica actúa como plataforma de interacción entre el usuario y los módulos de procesamiento digital permitiendo visualizar los resultados en los dominios del tiempo y la frecuencia.

PLANTEAMIENTO DEL PROBLEMA

En los laboratorios de electrónica, el análisis de señales en circuitos RLC suele realizarse manualmente, lo que implica mediciones visuales, cálculos posteriores y posibles errores de interpretación. Además, no siempre se dispone de equipos avanzados que permitan exportar datos para su análisis digital. Esto limita la capacidad de realizar estudios más profundos como el cálculo automático de parámetros característicos de la señal ni tampoco permiten la aplicación de técnicas de análisis espectral (FFT), filtrado digital, entre otros.

Adicionalmente, la falta de una interfaz de análisis integrada obliga a realizar el procesamiento de manera fragmentada, dificultando la interpretación dinámica del sistema.

De acuerdo con lo anterior, se plantea la siguiente pregunta de investigación:

¿Es posible desarrollar una interfaz gráfica en Python que integre la adquisición y el procesamiento digital de señales para la caracterización temporal y frecuencial de un circuito RLC?

JUSTIFICACIÓN

Este proyecto se justifica académicamente porque permite integrar conocimientos de:

- Electrónica analógica (circuitos RLC)
- Electrónica digital
- Sistemas embebidos
- Programación en Python
- Procesamiento digital de señales

Además, contribuye a la modernización de prácticas de laboratorio al automatizar el análisis experimental. Desde el punto de vista tecnológico, demuestra la viabilidad de sistemas de bajo costo para adquisición de datos.

MARCO CONCEPTUAL

Circuito RLC

Un circuito RLC es un sistema de segundo orden compuesto por una resistencia (R), una inductancia (L) y un capacitor (C). Su comportamiento puede ser subamortiguado, críticamente amortiguado o sobreamortiguado.

Frecuencia natural:

$$f_0 = \frac{1}{2\pi\sqrt{LC}}$$

Conversión Analógica-Digital

La ESP32 posee un ADC de 12 bits, que convierte señales analógicas (0–3.3V) en valores digitales entre 0 y 4095.

Comunicación Serial

La comunicación serial permite la transmisión de datos entre el Arduino y el computador mediante un protocolo basado en texto.

Procesamiento de Señales en Python

Se utilizarán las siguientes librerías:

- NumPy (cálculo numérico)
- Pandas (estructuración de datos)
- Matplotlib (visualización gráfica)
- Tkinter (Creación de interfaz)

Interfaz Gráfica de Usuario

Una interfaz gráfica permite la interacción directa entre el usuario y el sistema de procesamiento digital. En este proyecto facilita la configuración del sistema de adquisición, la ejecución de algoritmos de procesamiento digital de señales y la visualización simultánea de los resultados en los dominios del tiempo y frecuencia.

Muestreo

El muestreo consiste en la conversión de una señal continua en una señal discreta. Para evitar aliasing se debe cumplir con el criterio de Nyquist, el cual establece que la frecuencia de muestreo debe ser al menos el doble de la frecuencia máxima de la señal.

Transformada rápida de Fourier (FFT)

La FFT permite analizar el contenido espectral de una señal discreta, facilitando la identificación de la frecuencia máxima y la respuesta en frecuencia de un circuito RLC.

Filtrado digital

El filtrado digital se usa para reducir el ruido presente en una señal. Se puede implementar filtros FIR o IIR para mejorar la calidad de los datos.

Identificación de sistemas

A partir de la respuesta temporal del circuito, es posible estimar parámetros dinámicos como el sobreimpulso (M_p), tiempo de pico (t_p), frecuencia natural (ω_n) y el factor de amortiguamiento (ζ).

Estos se calculan con las siguientes formulas:

$$M_p = e^{-\frac{\zeta\pi}{\sqrt{1-\zeta^2}}}$$

$$\zeta = \frac{\omega_n}{2\omega_n}$$

$$t_p = \frac{\pi}{\omega_n\sqrt{1-\zeta^2}}$$

Los sistemas de segundo orden se clasifican en tres dependiendo del tiempo de amortiguamiento: subamortiguado cuando menor a 1, críticamente amortiguado cuando es igual a 1 y sobreamortiguado cuando es mayor a 1.

OBJETIVO GENERAL Y ESPECÍFICOS

- **Objetivo General**

Desarrollar un sistema con interfaz gráfica para la adquisición, visualización ya análisis digital de señales en un circuito RLC, utilizando ESP32 y procesamiento en Python para facilitar el aprendizaje de los estudiantes de los circuitos de segundo orden.

- **Objetivos específicos**

- Realizar una revisión del estado del arte relacionado con sistemas de adquisición de señales y análisis digital.
- Diseño y modelado teórico del circuito RLC
- Implementación del sistema de adquisición.
- Procesamiento digital de la señal.
- Desarrollar la interfaz
- Análisis, validación y documentación.

METODOLOGÍA

La metodología del proyecto se organiza en las siguientes fases acordes a los objetivos específicos:

Fase 1: Revisión del estado del arte.

- Actividad 1.1 – Investigar sobre sistemas de adquisición de datos en tiempo real.
- Actividad 1.2 – Revisión aplicaciones de procesamiento digital de señales.
- Actividad 1.3 – Análisis sobre interfaces graficas educativas para el análisis de sistema dinámicos

Fase 2: Diseño del sistema y modelado teórico

- Actividad 2.1 – Modelado teórico del circuito RLC.
- Actividad 2.2 – Selección de parámetros del circuito.
- Actividad 2.3 – Definición de condiciones de excitación.

Fase 3: Implementación del sistema de adquisición

- Actividad 3.1 – Implementación física del circuito.
- Actividad 3.2 – Programación del ESP32.
- Actividad 3.3 – Configuración de la comunicación serial.
- Actividad 3.4 – Programación en Python

Fase 4: Procesamiento digital de señales

- Actividad 4.1 – Preprocesamiento de la señal.
- Actividad 4.2 – Filtrado Digital.
- Actividad 4.3 – Análisis en frecuencia mediante FFT.
- Actividad 4.4 – Estimación de parámetros dinámicos.

Fase 5: Desarrollo de la interfaz grafica

- Actividad 5.1 – Diseño de la interfaz gráfica.

- Actividad 5.2 – Integración de módulos de procesamiento digital
- Actividad 5.3 – Implementación de la visualización temporal y frecuencial.
- Actividad 5.4 – Visualización e interacción.

Fase 6: Análisis, validación y documentación

- Actividad 6.1 – Comparación con el modelo teórico
- Actividad 6.2 – Visualización comparativa de resultados.
- Actividad 6.3 – Elaboración del informe final.

RESULTADOS

En esta sección se presentan los resultados de cada fase teniendo en cuenta las actividades realizadas en cada fase. Se inicia con la fase 1 que corresponde con la revisión del estado del arte.

- Actividad 1.1 – Investigar sobre sistemas de adquisición de datos en tiempo real.
En noviembre del 2025, los estudiantes de sexto semestre de ingeniería electrónica de la universidad Santiago de Cali realizaron un muestreo y modelamiento discreto de un circuito RC como proyecto final de la asignatura de señales y sistemas [1]. Este modelamiento se llevó a cabo usando el microcontrolador Arduino para llevar el registro de la carga del capacitor. Este es un buen ejemplo de un sistema de adquisición de datos en tiempo real.

Otro ejemplo de sistemas de adquisición de datos fue que, en octubre del 2025, varias personas provenientes de diversos sitios académicos en china colaboraron para el diseño de una arquitectura de adquisición de datos en tiempo real para un taller de forja basada en IoT y preprocesamiento de datos con el fin de desarrollar un sistema de gestión energética adecuado para un taller de forja [2].

- Actividad 1.2 – Revisión aplicaciones de procesamiento digital de señales.

En aplicaciones de procesamiento digital de señales se revisa un estudio que aborda el procesamiento digital de señales de corriente y voltaje en subestaciones digitales basadas en el estándar IEC 614850. Este evalúa el uso de técnicas de muestreo descendente (diezmación) en entidades de fusión, para reducir la carga computacional en dispositivos y sistemas digitales inteligentes [3]. El estudio da a entender la importancia del procesamiento digital de señales como una herramienta clave para asegurar el buen funcionamiento de sistemas eléctricos modernos.

- Actividad 1.3 – Análisis sobre interfaces graficas educativas para el análisis de sistemas dinámicos.

Chronopoulos (2024) presenta el diseño e implementación de un sistema OFDM flexible y reconfigurable basado en una plataforma de procesamiento digital de señales (DSP). El objetivo principal del trabajo es demostrar que es posible implementar un sistema de comunicaciones multicarrier adaptable mediante software, sin necesidad de modificar el hardware subyacente [4]. En ese contexto la interfaz se utiliza como una herramienta de apoyo para la visualización y análisis de las señales procesadas, haciendo posible la observación de parámetros como el número de subportadoras, el efecto del prefijo cíclico, el Zero padding y las variaciones en la potencia espectral.

A partir de los resultados obtenidos en la fase 1, se estableció un precedente o una base en cual podemos desarrollar el proyecto. Con esto, ya podemos pasar a la fase 2, que corresponde al diseño del sistema y modelado teórico.

- Actividad 2.1 – Modelado teórico del circuito RLC.

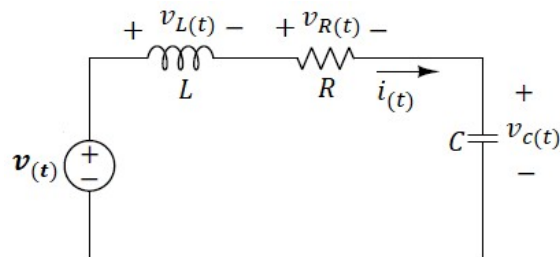


Ilustración 1 – Esquemático circuito RLC

Para el cálculo del circuito RLC que se muestra en la ilustración 1, se utilizó el método de malla, estableciendo la misma corriente que circula por cada componente y luego transformada de Laplace para hallar la función de transferencia de la salida de voltaje del capacitor. A continuación, se muestra el procedimiento:

$$IR + L \frac{dI}{dt} + \frac{1}{C} \int_0^t Idt = V_{in}$$

$$RI(S) + LSI(S) + \frac{I(S)}{CS} = V_{in}(S)$$

$$\left(R + LS + \frac{1}{CS} \right) I(S) = V_{in}(S)$$

$$G(S) = \frac{V_c(S)}{V_{in}(S)} = \frac{1}{LCS^2 + RCS + 1}$$

$$G(S) = \frac{\frac{1}{LC}}{S^2 + \frac{RC}{LC}S + \frac{1}{LC}}$$

La ecuación anterior corresponde a la función de transferencia con los parámetros eléctricos.

- Actividad 2.2 – Selección de parámetros del circuito.

Para el voltaje, se definió una magnitud de 1V para evitar que el sobre impulso del inductor dañe el pin analógico del microcontrolador ESP32 que funciona a 3.3V.

Para los componentes electrónicos, se tomó en cuenta las siguientes ecuaciones:

Función Canónica de segundo orden:

$$\frac{y(s)}{f(s)} = \frac{Kw_n^2}{s^2 + 2\zeta w_n s + w_n^2}$$

La anterior ecuación muestra la forma general de representar funciones de transferencias de segundo orden, el cual podemos ajustar los valores de sus polos según los componentes que utilicen, así que para poder lograr una respuesta subamortiguada, se utilizó la siguiente ecuación general:

$$R < 2\sqrt{\frac{L}{C}}$$

Esta ecuación, es la que ayuda a definir correctamente los valores de cada componente. Así que por ensayo y error se definió un inductor de 10mH y un capacitor de 100uF para tener una respuesta más lenta, el valor de R fue de 10Ω .

Con los valores mencionados anteriormente, la función de transferencia o modelo matemático que queda es el siguiente:

$$G(S) = \frac{1e + 6}{S^2 + 1000S + 1e + 6}$$

Calculando los parámetros dinámicos tenemos que:

$$\zeta = 0.5$$

$$\omega_n = 16.3\%$$

$$t_p = 0.00363s$$

- Actividad 2.3 – Definición de condiciones de excitación.

El circuito se excita con una señal escalón de magnitud 1, el cual tiene una duración muy corta de aproximadamente 500ms, el cual es suficiente para visualizar la señal.

Una vez establecido el modelo teórico del circuito RLC y que este responda de manera subamortiguado, se procedió a la fase 3, correspondiente a la implementación del sistema de adquisición.

- Actividad 3.1 – Implementación física del circuito.

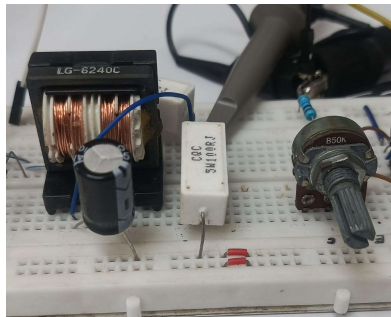


Ilustración 2 – Montaje del circuito

El circuito anterior se montó en la protoboard para realizar las pruebas, y se midió con el osciloscopio y el resultado se muestra en la siguiente imagen:

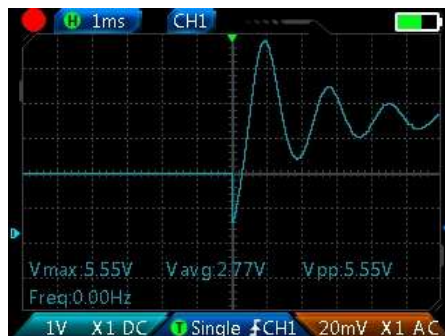


Ilustración 3 – Respuesta temporal de prueba

La imagen anterior muestra la respuesta del circuito ante una señal escalón, en este caso se había utilizado 5V en la alimentación debido a que se estaban haciendo pruebas con el osciloscopio y no directamente con el microcontrolador.

Se diseño el siguiente esquemático ya con todo lo necesario para su aplicación:

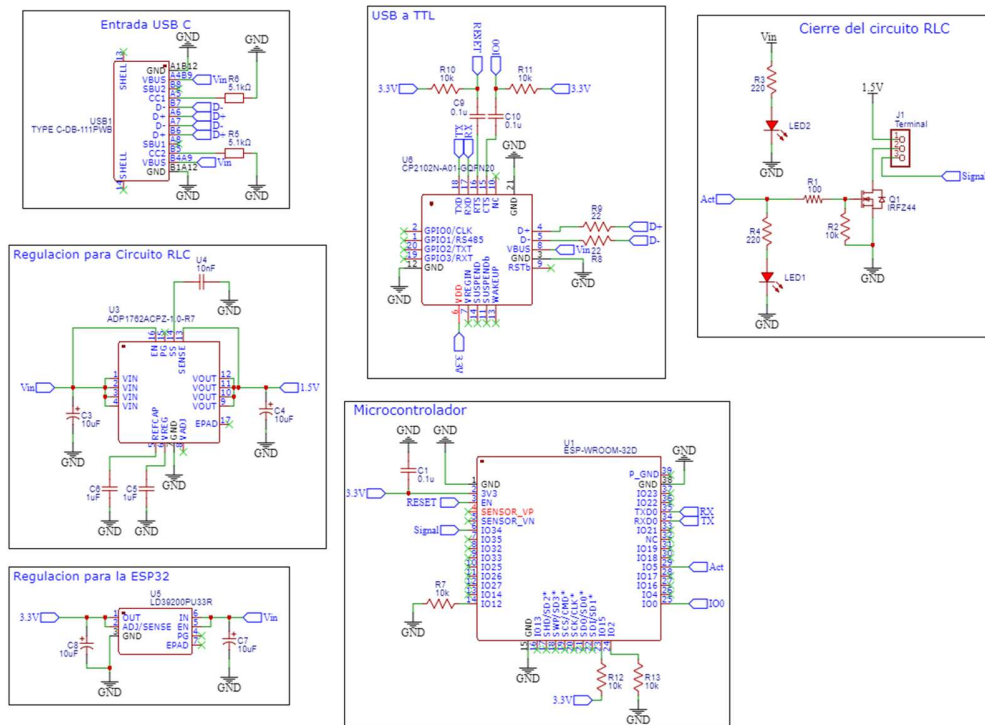


Ilustración 4 – Esquemático del microcontrolador

- Actividad 3.2 – Programación del ESP32.

```

int entrada = 39;

void setup() {
  // put your setup code here, to run once:
  pinMode(entrada, INPUT);
}

void loop() {
  // put your main code here, to run repeatedly:
  int valor = analogRead(entrada);
}

```

Ilustración 5 – Fragmento de código de la ESP32

La imagen anterior muestra cómo se lee el pin 39 de la ESP32 en modo análogo.

- Actividad 3.3 – Configuración de la comunicación serial.

```

int entrada = 39;

void setup() {
  // put your setup code here, to run once:
  Serial.begin(115200);
  pinMode(entrada, INPUT);
}

void loop() {
  // put your main code here, to run repeatedly:
  int valor = analogRead(entrada);
  Serial.println(valor);
  delay(500);
}

```

Ilustración 6– Fragmento de la comunicación serial de la ESP32

La imagen anterior muestra cómo se lee el pin y el envío de los valores al monitor serial cada 500ms.

- Actividad 3.4 – Programación en Python

La programación se llevó a cabo usando programación orientada a objetos o POO, para tener un código más ordenado.

Ya en python se creo el modulo de Adquisición_de_datos con la clase RLCSampler que tiene la funcion de conectar con la ESP32, mandar la

señal de activación al MOSFET, generando la señal escalón, capturar los datos generados y cerrar la conexión.

```
class RLCSampler:
    def __init__(self, puerto="COM4", baudios=460800, timeout=5): ...

    def conectar(self): ...

    def cerrar(self): ...

    def filtro_media_movil(self, data, n): ...

    def _validar_conexion(self): ...

    def capturar_paso(self, duracion=0.05, pre_delay=0.1, post_delay=0.2): ...
```

Ilustración 7 – Clase RLCSampler y Funciones

Con el sistema de adquisición implementado y funcionando correctamente se procedió a la fase 4, en la cual se aplica el procesamiento digital de la señal obtenida.

- Actividad 4.1 – Preprocesamiento de la señal.

La señal adquirida del ESP32 presenta ruido y un deslizamiento DC producto de la conversión analógica-digital. Por esa razón se decidió aplicar un preprocesamiento que consiste en un filtro y una normalización de la amplitud.

Se creó un modulo llamado data.py donde guardan los datos capturados en archivos .csc. En cual tiene la clase llamada DataIO, la cual tiene las siguientes funciones.

```
class DataIO:
    def __init__(self, folder="results/data"): ...

    def guardar_datos(self, t, y): ...

    def cargar_datos(self, path): ...
```

Ilustración 8– Clase DataIO y Funciones

Se creó una carpeta para toda la parte de procesamiento llamado Procesamiento.

- Actividad 4.2 – Filtrado Digital.

Se creó un modulo dentro de la carpeta de procesamiento llamado filtro.py. Este tiene la clase llamada Procesamiento_señal en el cual se diseñó un filtro tipo FIR pasa bajas Butterworth ya que este es ideal para aplicaciones de procesamiento digital de señales y también se diseñó la función de la normalización.

```
class Procesamiento_señal:
    def __init__(self, fs): ...

    def pasa_bajas(self, signal, fc=100): ...

    def normalizacion(self, signal): ...
```

Ilustración 9 – Clase Procesamiento_señal y Funciones

- Actividad 4.3 – Análisis en frecuencia mediante FFT.

Se creó un modulo fft.py para este apartado. Dentro de él se creó una clase llamada FFT_analizador.

```
class FFT_analizador:
    def __init__(self, fs): ...

    def fft_escalon(self, y, window=True, kernel=5): ...
```

Ilustración 10– Clase FFT_analizador y Funciones

Para la aplicación de la FFT se optimizó para la respuesta al escalón ya que si se aplica directamente la transformada generalmente da cero.

Aplicamos los siguientes pasos:

1. Aplicamos derivación discreta para eliminar el nivel DC.
2. Aplicamos una ventana tipo Hanning para reducir el spectral leakage.
3. Aplicamos la transformada.
4. Aplicamos un suavizado a la señal.

- Actividad 4.4 – Estimación de parámetros dinámicos.

Se creó un módulo llamado parametros_dinamicos.py. En el se creó la clase llamada Identificar_sistema. En el se creó la función para verificar si la señal capturada es de segundo orden.

```
class Identificar_sistema:
    def __init__(self, t, y): ...

    def verif_segundo_orden(self): ...
```

Ilustración 11 – Clase Identificar_sistema y Funciones

Donde se aplicaron las fórmulas para calcular factor de amortiguamiento (ζ), el sobreimpulso (M_p), tiempo de pico (t_{peak}) y frecuencia natural (ω_n). Con esto estimamos si es de segundo orden.

Se creó otro modulo llamado fdt.py con la clase llamada Estimador_FDT para tener la forma canónica de las ecuaciones de segundo orden.

```
class Estimador_FDT:
    def __init__(self, zeta, wn): ...

    def Obtener_funcion_transferencia(self): ...
```

Ilustración 12 – Clase Estimador_FDT y Funciones

Con los resultados anteriores de procesamiento digital se pudo caracterizar el comportamiento dinámico del sistema, con esto, se procede a la fase 5, la creación de la interfaz gráfica en la cual mostraremos todo lo obtenido en la fase 4.

- Actividad 5.1 – Diseño de la interfaz gráfica.

Para la interfaz se optó por un diseño que fuera muy intuitivo y de fácil comprensión, orientado a facilitar la interacción del usuario con el sistema y la interpretación de resultados. La ventana principal se dividió en 4 partes, cada una con su función:

- La gráfica, destinada a la visualización de la respuesta del sistema.
- La función de transferencia, donde se presenta el modelo matemático que describe el comportamiento del sistema.
- Los controles, encargados de la configuración de los parámetros del sistema.
- Los botones, que permiten ejecutar las acciones principales del programa.

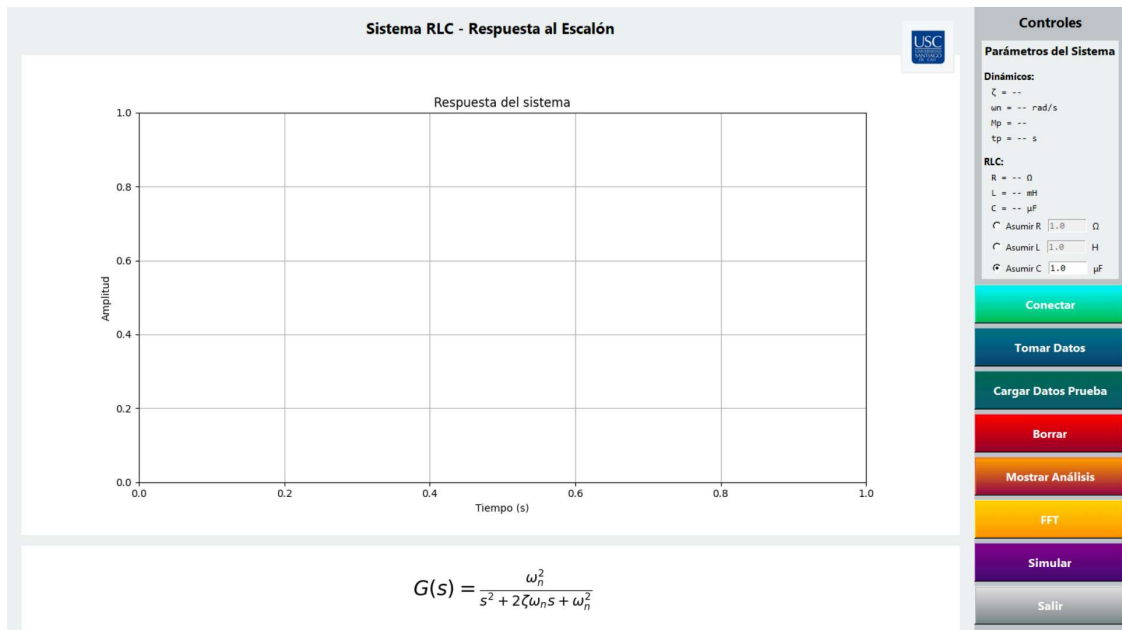


Ilustración 13 – Interfaz Completa

- Actividad 5.2 – Integración de módulos de procesamiento digital

Los módulos de procesamiento desarrollados en la fase 4 se integraron mediante los botones.

 - El botón de conectar verifica que la ESP32 este conectada correctamente.
 - El botón de tomar datos manda la señal de activación al MOSFET y almacena los datos capturados.
 - El botón de cargar datos de prueba carga los datos de prueba generados anteriormente para verificar el funcionamiento del sistema.
 - El botón de borrar limpia la gráfica y los datos almacenados.
 - El botón de mostrar análisis aplica la fase de procesamiento de la señal (filtrado y normalización) a la señal graficada y calcula los parámetros dinámicos.
 - El botón de FFT aplica la transformada de Fourier a la señal graficada y la muestra.
 - El botón de simular permite ver como seria la respuesta al escalón con valores R, L y C que el usuario desea. También tiene un apartado para seleccionar el tipo de respuesta que desean visualizar y este coloca los valores con los que da la respuesta seleccionada.
 - El botón de salir cierra el programa.
- Actividad 5.3 – Implementación de la visualización temporal y frecuencial.

Se implementó la visualización de la señal tanto en el dominio del tiempo como en el dominio de la frecuencia. En el dominio temporal se muestra la respuesta al escalón del sistema RLC con el botón de tomar datos, si son datos muestreados, con el botón de cargar datos de prueba, si son datos generados anteriormente o con el botón de simular. Así mismo, con el botón de FFT, para analizar el contenido espectral se la señal graficada.

- Actividad 5.4 – Visualización e interacción.

Se desarrollaron varios apartados con mecanismos interacción usuario-sistema que permiten una experiencia más inmersiva.

- Para el cálculo de la función de transferencia a partir de los datos muestreados se necesita conocer al menos un valor de los componentes del sistema, ya sea el de la resistencia, la capacitancia o la inductancia. Ya con estos se puede aplicar las fórmulas correspondientes para la obtención de la función de transferencia. Debido a esto, se creó un apartado para que el usuario decida e ingrese el valor del componente que conoce. Este valor solo se tiene en cuenta si son datos muestreados o datos de prueba.
- También se creó un apartado en la parte de simulación el cual permite ya sea ingresar los valores de R, L o C que el usuario desea o bien escoger el modo de respuesta de los que están ahí y visualizarlo. Los modos de respuesta añadidos son: subamortiguado (este viene por defecto), críticamente amortiguado y sobreamortiguado.
- Posee también, un apartado que permite visualizar los parámetros dinámicos de los datos graficados, ya sean muestreados en tiempo real, datos de prueba o datos simulados.

Una vez integrada la fase 4 y la fase 5, pasamos a la fase 6 correspondiente a la parte del análisis y validación con el modelo teórico y la finalización del informe final.

- Actividad 6.1 – Comparación con el modelo teórico

Anteriormente se definió la siguiente función de transferencia:

$$G(S) = \frac{1e + 6}{S^2 + 1000S + 1e + 6}$$

Esta tiene un comportamiento subamortiguado ya que $\zeta < 1$.

Simulando con la interfaz tenemos la siguiente función de transferencia:

$$G(s) = \frac{1000000.0}{s^2 + 1000.0s + 1000000.0}$$

Ilustración 14 – Función de transferencia simulación

Y los parámetros dinámicos calculados automáticamente:

Parámetros del Sistema	
Dinámicos:	
ζ	= 0.5000
ω_n	= 1000.00 rad/s
M_p	= 16.297
t_p	= 0.0036 s
RLC:	
R	= 10.0 Ω
L	= 10.0 mH
C	= 100 μF

Ilustración 15– Parámetros Dinámicos Simulación

Tomando los datos en tiempo real tenemos lo siguiente:

$$G_{est}(s) = \frac{11800.6}{s^2 + 17.3s + 11800.6}$$

Ilustración 16– Función de transferencia de datos muestreados

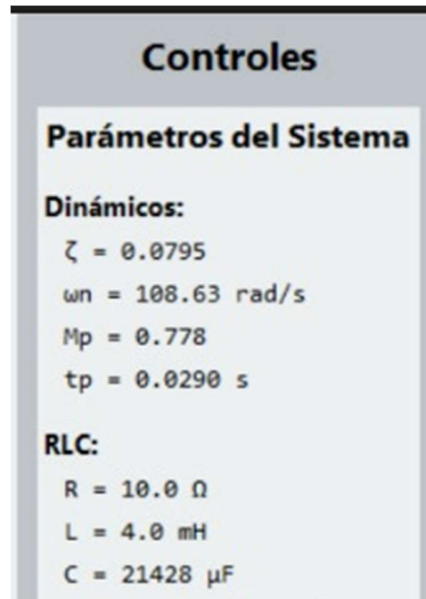


Ilustración 17 – parámetros dinámicos de datos muestreados

Como se pudo observar anteriormente, los resultados de la simulación y muestreo concuerdan con el modelo teórico, verificando en funcionamiento correcto del programa. Las diferencias mostradas en lo muestreo se debieron principalmente al ruido.

- Actividad 6.2 – Visualización comparativa de resultados.

Simulando tenemos las siguientes respuestas:

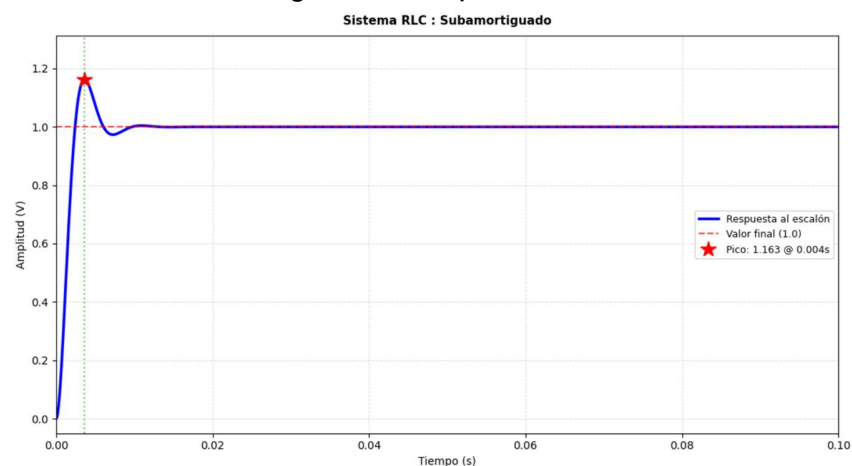


Ilustración 18– Respuesta temporal simulación

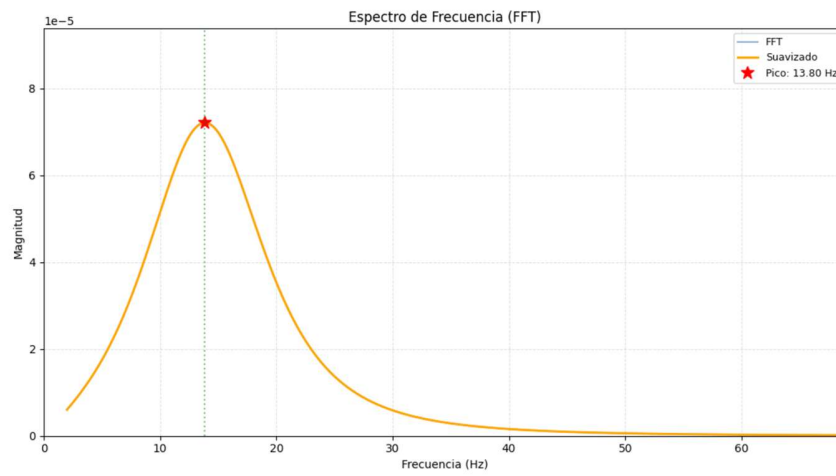


Ilustración 19– Respuesta espectral simulación

Tomando los datos en tiempo real tenemos las siguientes respuestas:

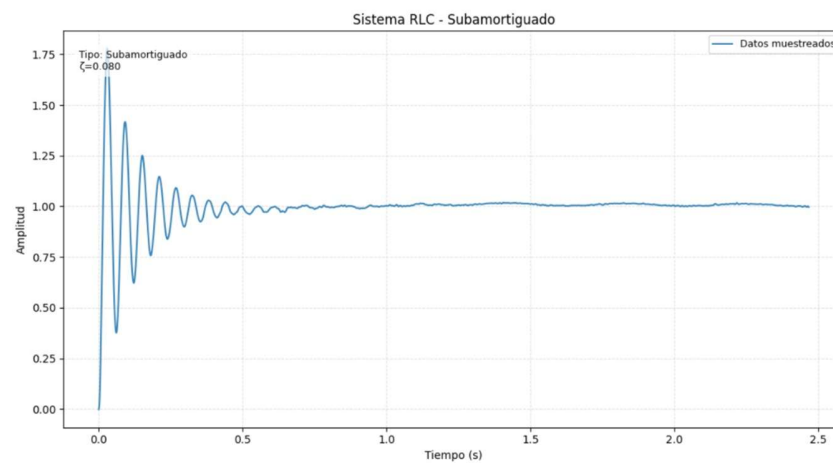


Ilustración 20 – Respuesta temporal de datos muestreados

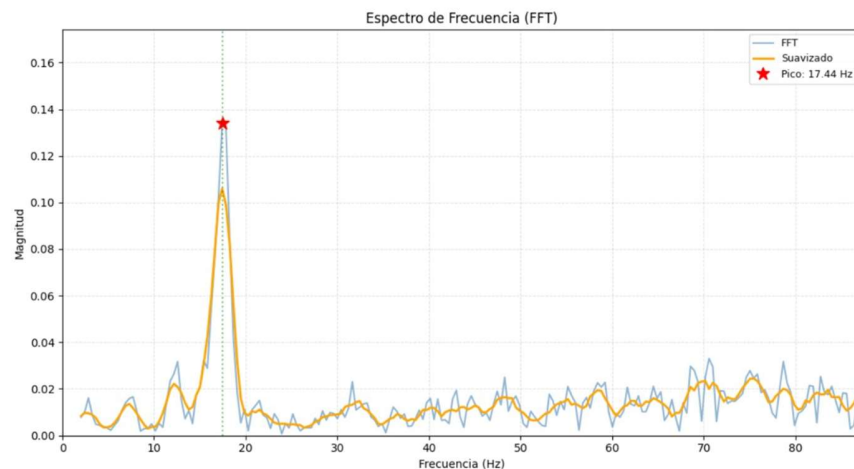


Ilustración 21 – Respuesta espectral de los datos muestreados

Anteriormente, se presentaron las gráficas de las repuestas temporal y espectral de la simulación y los datos muestreado en tiempo real, esta visualización permitió identificar sus diferencias y validar la coherencia con su modelo teórico.

- Actividad 6.3 – Elaboración del informe final.

Finalmente, la ejecución del proyecto por fases permitió llegar al objetivo del proyecto, que es el desarrollo de una interfaz gráfica para el análisis de sistemas de segundo orden con el fin de facilitar el aprendizaje de los estudiantes de estos sistemas.

POSIBLES MEJORAS Y TRABAJO FUTURO

El sistema desarrollado cumple con el objetivo de integrar la adquisición, el procesamiento digital y la visualización de señales en un circuito RLC. No obstante, a partir de los resultados obtenidos y de la experiencia durante el desarrollo del proyecto, se identifican diversas oportunidades de mejora que permitirían aumentar la robustez, precisión y escalabilidad del sistema en futuras implementaciones.

Una posible mejora propuesta es el diseño y la implementación de una placa de circuito impreso (PCB) que integre el circuito RLC junto con el sistema de adquisición basado en el microcontrolador. Actualmente el circuito se desarrollo sobre protoboard, lo cual puede inducir ruido eléctrico, variaciones en la conexiones y limitaciones en la repetibilidad de las mediciones. Por ello se propone la siguiente PCB desarrollada en el software EasyEDA de JLCPCB:

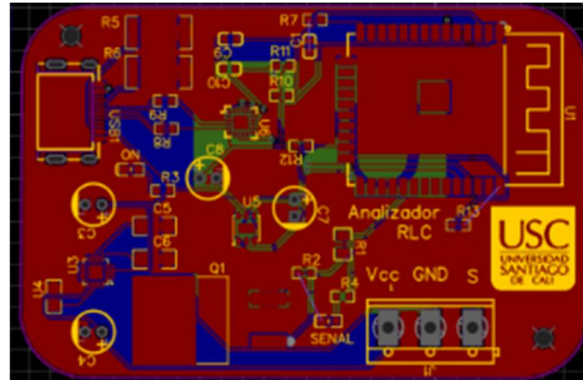


Ilustración 22 – Diagrama de conexiones de la PCB



Ilustración 23 – PCB en 2D

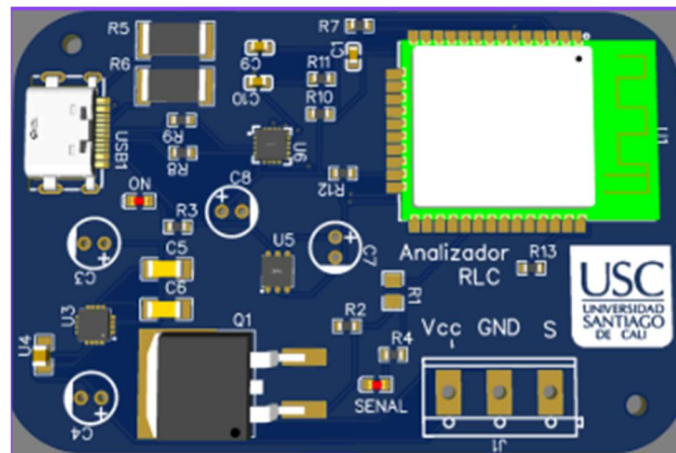


Ilustración 24 – PCB Final

La implementación de una PCB permitiría:

- Reducir interferencias y ruido asociado a conexiones sueltas.
- Mejorar la estabilidad eléctrica del sistema.
- Proteger los pines del microcontrolador.
- Facilitar la replicación del sistema para uso académico y de laboratorio.

CRONOGRAMA

El cronograma se hizo en modelo de semanas. Se inicio el jueves 19 de febrero del 2026 y se finalizó el jueves 14 de mayo del 2026 comprendido en 12 semanas.

Semana	Fechas	Fase	Actividad	Progreso
1	Del 19 de febrero de 26 de febrero	Fase 1	Actividad 1.1 – Investigar sobre sistemas de adquisición de datos en tiempo real	100%
		Fase 1	Actividad 1.2 – Revisión de aplicaciones de procesamiento digital de señales	100%
		Fase 1	Actividad 1.3 – Análisis sobre interfaces gráficas educativas para sistemas dinámicos	100%
2	Del 26 de febrero al 5 de marzo	Fase 2	Actividad 2.1 – Modelado teórico del circuito RLC	100%
		Fase 2	Actividad 2.2 – Selección de parámetros del circuito	100%
		Fase 2	Actividad 2.3 – Definición de condiciones de excitación	100%
3	Del 6 al 12 de marzo	Fase 3	Actividad 3.1 – Implementación física del circuito	100%
4	Del 13 al 19 de marzo	Fase 3	Actividad 3.2 – Programación del ESP32	100%
		Fase 3	Actividad 3.3 – Configuración de la comunicación serial	100%
5	Del 20 al 26 de marzo	Fase 3	Actividad 3.4 – Programación en Python	100%
	Del 27 de marzo	Fase 4	Actividad 4.1 – Preprocesamiento de la señal	100%
		Fase 4	Actividad 4.2 – Filtrado digital	100%

6,7 y 8	al 16 de abril	Fase 4	Actividad 4.3 – Análisis en frecuencia mediante FFT	100%
		Fase 4	Actividad 4.4 – Estimación de parámetros dinámicos	100%
		Fase 5	Actividad 5.1 – Diseño de la interfaz gráfica	100%
		Fase 5	Actividad 5.2 – Integración de módulos de DSP	100%
9	Del 17 al 23 de abril	Fase 5	Actividad 5.3 – Visualización temporal y frecuencial	100%
10 y 11	Del 24 de abril al 7 de mayo	Fase 5	Actividad 5.4 – Visualización e interacción	100%
		Fase 6	Actividad 6.1 – Comparación con el modelo teórico	100%
		Fase 6	Actividad 6.2 – Visualización comparativa de resultados	100%
12	Del 8 al 14 de mayo	Fase 6	Actividad 6.3 – Elaboración del informe final	100%

PRESUPUESTO

El presente presupuesto corresponde a una estimación académica del proyecto, considerando el tiempo de dedicación estudiantil, los materiales utilizados y los recursos necesarios para su desarrollo.

1. Costos de materiales y hardware.

Ítem	Descripción	Cantidad	Valor unitario (COP)	Valor total (COP)
ESP32	Módulo de desarrollo	1	40.000	40.000
Resistencias	Kit para circuito RLC	1	10.000	10.000

Capacitores	Electrolíticos y cerámicos	1	15.000	15.000
Inductor	Bobina para RLC	1	20.000	20.000
Protoboard	Montaje del circuito	1	15.000	15.000
Cables jumper	Conexiones del sistema	1	10.000	10.000
Fuente DC	Fuente de alimentación	1	25.000	25.000
Subtotales materiales				135.000

2. Costos de trabajo académico

Tarifa académica estimada: 12.000 COP / hora.

Dedicación: 5 horas/semana por estudiante durante 12 semanas.

Actividad	Integrantes	Horas totales	Valor (COP)
Modelado teórico y análisis DSP	1	60	720.000
Implementación hardware y programación ESP32	1	60	720.000
Desarrollo de interfaz gráfica y análisis de resultados	1	60	720.000
Subtotal trabajo académico	3	180	2.160.000

3. Costos de apoyo académico

Incluye uso de equipos personales, energía eléctrica, acceso a internet y recursos computacionales necesarios para el desarrollo del proyecto.

Concepto	Valor estimado (COP)
Apoyo académico y recursos tecnológicos	200.000

4. Costos de documentación

Ítem	Descripción	Valor (COP)
Impresión del informe final	Documento académico	10.000
Encuadernación	Presentación final	10.000

Subtotal documentación	20.000
-------------------------------	---------------

5. Resumen general de presupuesto.

Concepto	Valor (COP)
Materiales y hardware	135.000
Trabajo académico (estudiantes)	2.160.000
Apoyo académico	200.000
Documentación	20.000
Costo total estimado del proyecto	2.515.000 COP

Bibliografía

- [1] J. A. Cuellar, L. J. Pinto, J. M. Gonzalez y A. Nazarith, «Muestreo y modelo discreto de un circuito RC usando Arduino,» Santiago de Cali, 2025.
- [2] Y. Luo, L. Li, L. Li, H. Zhang y C. Li, «Real-time acquisition, pre-processing and mining of energy consumption,» China, 2025.
- [3] A. Kulikova, P. Llyushin, K. Suslov y D. Karamov, «Coherence of digital processing of current and voltage signals at,» Moscow, Russia, 2022.
- [4] S. K. Chronopoulos, "Flexible and Reconfigurable OFDM Implementation in DSP Platform for Various Purposes and Applications," *Sensors*, vol. 9, no. 2732, p. 24, 2024.